Timeouts as a Resilience Tool

a slow dependency is worse than a dead one

Watch the 4-min video

In one line: every remote call needs an explicit connect and request timeout, set a little above the dependency's p99 latency and shorter than the caller's own timeout, so a hanging dependency fails fast instead of holding threads until the whole service stops.

1 It didn't fail. It just stopped answering

The payment provider doesn't return errors: it accepts connections and never replies. Your checkout service waits, and waits. Within a minute every worker thread is stuck on a payment call, and the product page, which never touches payments, stops loading too. One missing timeout turned a slow dependency into a full outage.

2 What a timeout bounds

Timeout	Bounds	Typical value
Connect	establishing the TCP (and TLS) connection	short: 100 ms – 1 s inside a data centre
Read / socket	gap between bytes of the response	a little above the dependency's p99
Request (total)	whole call, including retries	fits inside the caller's budget
Deadline	absolute time the whole request must finish by	set at the edge, propagated inward

Defaults are dangerous. Python's `requests` and Go's `http.Client` wait forever unless you set a timeout, and an unanswered TCP connect on Linux takes about two minutes to give up. Always set them explicitly.

3 Why hanging calls take everything down

Every waiting request holds a thread, a connection, memory and perhaps a database transaction. By Little's law, threads in use = arrival rate × time each request takes. At 100 requests/s and 50 ms per call you need about 5 threads; if the call hangs for 30 s you'd need 3,000. Once the pool is full, new requests queue and fail, including requests for healthy endpoints. A **dead** dependency fails fast with "connection refused"; a **slow** one is more dangerous.

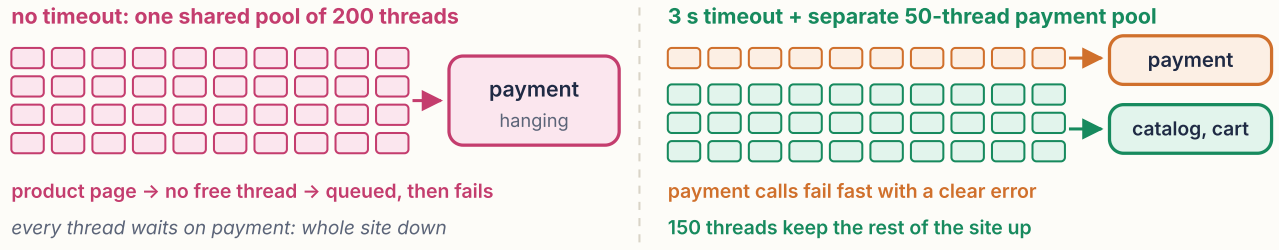


Figure 1. Left: hanging calls fill the shared pool. Right: a timeout bounds the wait and a bulkhead (lesson 7) bounds how many threads payments can hold.

4 Choosing values and layering them

Base the value on **measured latency**: a little above the dependency's p99 or p99.9. Too long and threads stay stuck; too short and you fail requests that would have succeeded, and trigger extra retries that add load. Review the values when latency changes.

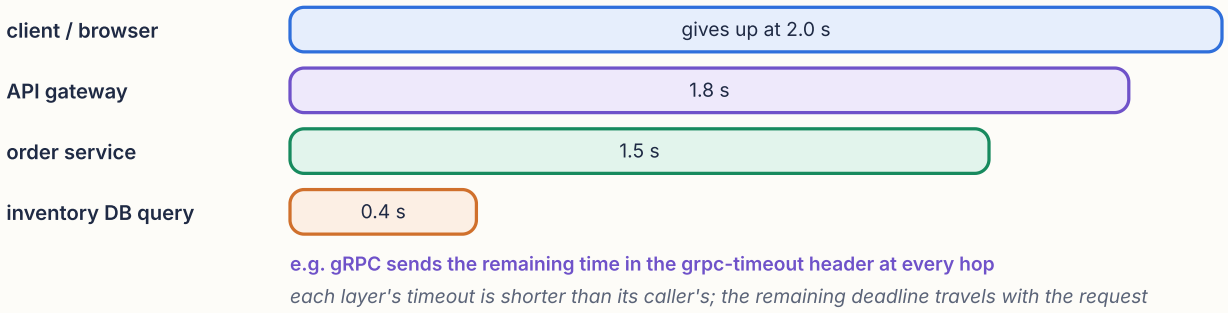


Figure 2. A timeout budget: work after the caller has given up is wasted, so inner timeouts must be shorter.

If the browser gives up after 2 s, the service shouldn't wait 10 s for a downstream call. Better than fixed values is a **propagated deadline**: the edge sets "finish by 12:00:02.000", and each hop checks the remaining time before starting work and passes it on. gRPC does this natively with deadlines.

WORKED EXAMPLE: A HANGING PAYMENT PROVIDER

A checkout service has **200 worker threads** and no timeout on its payment client. The provider starts hanging; in under a minute all 200 threads are blocked and the product page stops loading. Fix: `connect_timeout=0.5, timeout=3.0` plus a separate **payment pool of 50 threads**. At 10 payment calls/s, even if every one hangs for the full 3 s, only about 30 threads are busy. Payment fails fast with "payment temporarily unavailable", and the other 150 threads keep the site up.

TIMEOUTS WORK WITH OTHER PATTERNS

- **Retries** (lesson 5): only for idempotent calls, and counted inside the overall deadline.
- **Circuit breaker** (lesson 6): stop calling a dependency that keeps timing out.
- **Bulkhead** (lesson 7): a slow dependency can only exhaust its own pool.
- **Fallback** (lesson 8): decide what to return when the timeout fires.

COMMON MISTAKES

- Relying on library defaults, which are often infinite or minutes long.
- Setting only a connect timeout, so a slow response still hangs forever.
- Inner timeouts longer than outer ones, so work continues after the user has gone.
- Timeouts set once and never revisited as latency changes.

5 Key terms

Connect timeout: limit on establishing the connection

Request timeout: limit on getting the whole response

Deadline: absolute finish-by time passed along the call chain

Timeout budget: inner timeouts shorter than outer ones

Thread pool exhaustion: all workers blocked, new requests can't run

Little's law: in-flight requests = arrival rate × latency

6 Interview questions

Q1. What is the difference between a connect timeout and a request timeout? BASIC
The connect timeout bounds establishing the connection; the request (or read) timeout bounds waiting for the response once connected. You need both.

Q2. How do you choose a timeout value? BASIC
From measured latency: a little above the dependency's p99 or p99.9, and within the caller's remaining budget.

Q3. What happens to a service with no timeouts when a dependency hangs? SCENARIO
Requests pile up holding threads, connections and memory until the pools are exhausted; then the whole service stops responding, even for requests that never touch that dependency.

Q4. How should timeouts be set across layers? SCENARIO
As a budget: each layer's timeout shorter than its caller's, ideally with a deadline propagated with the request, as gRPC deadlines do.

Q5. Why is a slow dependency more dangerous than a dead one? TRICKY
A dead one fails immediately and frees resources; a slow one holds threads and connections for the full wait, so it exhausts pools and spreads the failure upstream.

Q6. Can a timeout that's too short hurt reliability? TRICKY
Yes: it fails requests that would have succeeded, and the retries it triggers add load to a dependency that's already slow.

Next: SD-K-005 · Retries and Backoff backendroadmap.com